

CSCE 711 APPLIED CRYPTOGRAPHY · MODULE 01 · LECTURE NOTES

What Cryptography Provides

And the rule that follows from it.

MODULE OUTCOMES

Name the four properties and distinguish them.

State what cryptography does not provide.

Explain how security is defined against an adversary.

State the course's governing rule and why it holds.

Identify which property a requirement actually needs.

1 The four properties

Property	The guarantee	Provided by
Confidentiality	The adversary learns nothing about the content beyond its length.	Encryption (Module 03).
Integrity	Any modification of the message is detected.	A message authentication code or a signature (Modules 04 and 07).
Authentication	The message originated from the party you believe it did.	The same mechanisms — see the note.
Freshness	This is not a valid old message being replayed.	Nonces, counters, timestamps, or a challenge — and it requires protocol-level thought rather than a primitive.

Integrity and authentication are nearly the same mechanism and are not the same property. A MAC gives you both at once, which is why they are frequently conflated — but **a checksum or CRC gives you neither**, because an adversary who modifies the message simply recomputes it. **And freshness is the one most often forgotten entirely**: a correctly authenticated message, captured and replayed an hour later, is still valid unless the protocol was designed to reject it.

ENCRYPTION ALONE PROVIDES NONE OF THE OTHER THREE

A ciphertext can be modified by an adversary in transit, and unauthenticated decryption will happily produce *different plaintext* without any complaint — the receiver has no way to tell.

With a stream cipher the relationship is exact: flipping a ciphertext bit flips exactly the corresponding plaintext bit, because the ciphertext is the plaintext XORed with a keystream. So an adversary who knows the plaintext format — which is usually public — can make targeted, predictable changes without possessing the key at all.

So 'it is encrypted' says precisely nothing about whether the data has been tampered with, and a system that decrypts unauthenticated data is processing attacker-controlled input while believing it is processing its own trusted data — which is a far worse position than processing input it knows to be untrusted.

Which is why authenticated encryption is the only acceptable default (Module 04) — and why this appears in §1 rather than as a later refinement, because every subsequent module assumes it.

2 What cryptography does not provide

- **It does not protect metadata.** Who communicated with whom, at what times, how often, and in what volumes — all of it visible, and frequently more revealing than the content. See the note.
- **It does not protect data in use.** The plaintext exists in memory in whatever process handles it, which is exactly where a compromised server reads it — so encryption at rest and in transit leaves the window where the data is actually being used.
- **It does not establish trust.** A valid signature proves that a particular key signed the message, not that the holder of that key is who they claim to be — which is Module 07's certificate problem and is not a cryptographic problem at all.
- **It does not fix authorisation.** A correctly authenticated request from a user who should not be permitted to make it is still a vulnerability (CSCE 701 Module 03) — authentication answers who, and authorisation answers whether.
- **And it does not survive bad key management**, which is where the overwhelming majority of real failures live (Module 10, and CSCE 701 Module 07). **The metadata point is the one most often missed:** a messaging system that encrypts every message body while leaking the complete social graph protects very little against an adversary whose objective was the graph.

3 Defining security

A SCHEME IS SECURE WHEN A SPECIFIED ADVERSARY CANNOT WIN A SPECIFIED GAME

'Secure' on its own is not a claim at all. A definition names three things: the adversary's capabilities, the adversary's goal, and the success probability considered acceptable. Change any one and you have a different claim.

So for encryption, the standard definition is indistinguishability under chosen-plaintext attack: an adversary who chooses two equal-length plaintexts and receives the encryption of one of them cannot tell which, appreciably better than guessing.

And that definition is considerably stronger than it first looks: it implies the adversary learns no partial information whatsoever — not a single bit, not whether two ciphertexts encrypt the same message, nothing beyond what the length itself discloses. Any scheme leaking even one bit of the plaintext fails it, which is why the bar is set there.

Which is why the framing is worth the effort: it converts 'this seems hard to break' into a precise statement that can be proved relative to a stated computational assumption — and the assumption then becomes the thing to examine, which is CSCE 637's territory and is where the post-quantum question lives (Module 12).

$$\text{Adv}(A) = | \Pr[A(\text{Enc}(m_0)) = 0] - \Pr[A(\text{Enc}(m_1)) = 0] |$$

- **A** — the adversary: any algorithm at all, within a stated bound on its running time.
- **m_0, m_1** — two plaintexts of equal length, chosen by the adversary with full knowledge of the scheme.
- **$\text{Adv}(A)$** — the advantage: how much better than a coin flip the adversary distinguishes the two cases.
- **Secure** — $\text{Adv}(A)$ is negligible for every efficient A . **Security is a bound on an advantage rather than an absolute property**, and 'efficient' and 'negligible' are the two terms doing the real work — they are what tie the definition to CSCE 637's complexity assumptions.

4 The rule

DO NOT DESIGN PRIMITIVES, AND DO NOT ASSEMBLE PROTOCOLS FROM PRIMITIVES

Use a reviewed high-level library. **libsodium, Google Tink, or your platform's audited equivalent** — and **prefer whichever API gives you the fewest choices to make**, because each choice is an opportunity to choose wrongly.

The reason is the historical record rather than caution: nearly every published break of a deployed system was a composition error or an implementation flaw rather than a broken primitive — a reused nonce, a padding oracle, a missing tag verification, an unvalidated curve point, a timing leak. **AES and SHA-256 have withstood decades of concentrated attack; the systems built on them have not.**

And those errors are essentially invisible to testing. A misused construction encrypts and decrypts correctly, round-trips perfectly, and passes every functional test you would think to write — which is precisely why the defect survives into production and is found by an adversary rather than by CI.

So the rule holds — and this course spends thirteen modules on why it holds, because **a rule you understand you can apply with judgement at the edge cases, and a rule you have merely been handed you will eventually break while believing you are the exception.**

- **Choose the library, not the algorithm. A good library has already made the algorithm, mode, padding, and parameter decisions** on the basis of expert review, and every one of those you make yourself is a chance to get it wrong.
- **Prefer misuse-resistant APIs. An interface that generates its own nonce internally cannot have its nonce reused by its caller** (Module 11 §2) — the whole class of failure is removed by the API shape rather than by discipline.
- **Treat every low-level API as a warning sign. If you are choosing an IV, selecting a padding mode, or calling a bare block cipher, you are in the wrong abstraction** — and this is a reliable test you can apply to code in review.
- **And if your requirement genuinely has no library answer, that is a signal to re-examine the requirement** before it is a signal to build something novel. **The requirement is usually the thing that is wrong.**
- **Project 1 has you implement a primitive precisely so that you never need to again** — and its deliverable explicitly includes the argument for not using what you built, written from what you found while building it.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Aumasson — <i>Serious Cryptography</i> , chapters 1 and 2	§1 and §3 at this course's level , and the treatment of what security definitions are for is the clearest available.

Resource	What it covers for this module
Boneh & Shoup — A Graduate Course in Applied Cryptography, chapter 2 (free)	§3 rigorously — the definitions, the advantage formulation, and what a security proof actually establishes.
Cryptographic Right Answers (free)	§4 as a practical list , maintained by people who review real systems for a living. Short, and worth rereading after Module 11.
The libsodium documentation (free)	§4's rule embodied in an API — read what it refuses to let you do, which is the design argument.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Take a system you use** and state which of the four properties it needs for each kind of data it handles.
- 2 **Find one place where integrity matters and confidentiality does not**, and one where the reverse holds.
- 3 **Encrypt a known plaintext with a stream cipher**, flip one ciphertext bit, decrypt, and observe the result.
- 4 **Predict which plaintext bit will change before you run it**, and check.
- 5 **List the metadata your own application leaks** even with perfect encryption.
- 6 **Write the indistinguishability game** in your own words, naming the adversary's powers.
- 7 **Explain why a scheme leaking one plaintext bit fails the definition**, and why that is the right bar.
- 8 **Find three low-level cryptographic API calls** in code you have access to, and say what the high-level replacement would be.
- 9 **Apply the IV test** to a piece of code and record the verdict.
- 10 **Write the rule in your own words**, with the reason, and keep it for Module 13.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the four properties and what each guarantees.
- 2 Why are integrity and authentication distinct, and what gives neither?
- 3 Why does encryption alone provide none of the other three?
- 4 What exactly happens when you flip a stream cipher ciphertext bit?
- 5 Give five things cryptography does not provide.
- 6 Why is the metadata point the one most often missed?
- 7 How is security defined, and what three things does a definition name?

- 8 What does indistinguishability imply about partial information?
- 9 State the course's rule and the historical reason for it.
- 10 Why are misuse errors invisible to testing?